



Introduction

- RNN were introduced in the late 80's.
 - Hochreiter discovers the 'vanishing gradients' problem in 1991.
- Long Short Term Memory published in 1997.
 - LSTM: a recurrent network to overcome this problem.

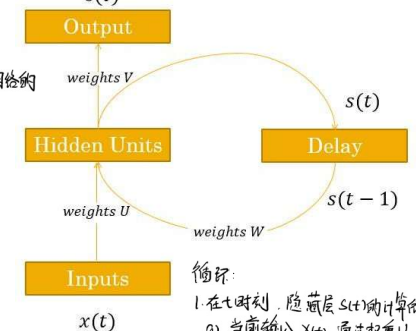
Motivation

- Feed forward networks
 - accept a fixed-sized vector as input
 - produce a fixed-sized vector as output
 - usually fixed amount of computational steps
- Recurrent networks allow us to operate over sequences of vectors
 - the idea behind RNNs is to make use of sequential information, as in a traditional neural network all inputs (and outputs) are assumed independent of each other
 - but for many tasks that's a very bad idea: if you want to predict the next word in a sentence you better know which words came before it

Elman 是 RNN 家族中最经典的一种

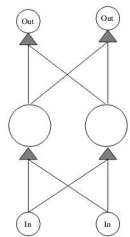
RNN Architecture

- RNN 对序列中的每一个元素, 执行“相同的任务”
它的输出不仅取决于“当前的输入”, 还取决于“上一步的计算结果”——这就是“记忆”(memory)
- Input (x): 当前时刻 (t 时刻) 的输入
- Hidden unit ($s(t)$): 当前时刻的隐藏状态, 网络的短期记忆, 或上下文
- Output ($o(t)$): 当前时刻的输出
- RNNs are called recurrent because they perform the same task for every element of a sequence, with the output being depended on the previous computations (memory).
 - inputs $x(t)$
 - outputs $y(t)$
 - hidden state $s(t)$: memory of the network
 - a delay unit is introduced to hold activation until they are processed at the next step
 - The decision a recurrent net reached at time step $t-1$ affects the decision it will reach one moment later at time step t
 - two sources of input: the present and the recent past, which combine to determine how they respond to new data
- 循环:
- 在 t 时刻, 隐藏层 $s(t)$ 的计算依赖于:
 - 当前输入 $x(t)$, 通过权重 U .
 - 上一时刻的记亿 $s(t-1)$, 通过权重 W .
 - $s(t-1)$ 就是 $s(t)$ 经过一个延迟单元之后反馈给旧记忆.

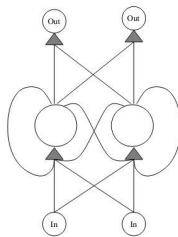


RNN是一种系列，其结构特点是“至少有一个循环连接”，Elman是最实用和简单的。

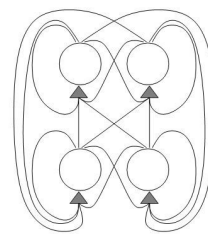
RNN Architecture



feed forward NN
RBF 或 MLP



a simple RNN
Elman

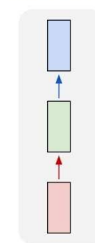


fully connected RNN

- RNN topologies range from partly recurrent to fully recurrent
 - partly recurrent is a layered network with distinct input and output layers where the recurrence is limited to the hidden layer
 - in fully recurrent networks each node gets inputs from all other nodes

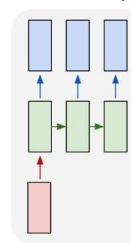
RNN Architecture

one to one



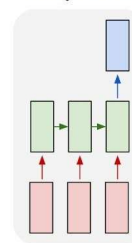
Vanilla mode of processing without RNN, from fixed-sized input to fixed-sized output (e.g. image classification).

one to many



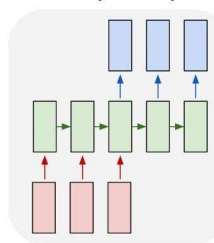
Sequence output (e.g. image captioning takes an image and outputs a sentence of words).

many to one



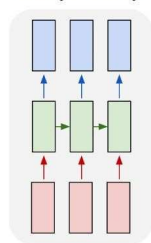
Sequence input (e.g. sentiment analysis where a given sentence is classified as expressing positive or negative sentiment)

many to many



Sequence input and sequence output (e.g. machine translation: an RNN reads a sentence in English and then outputs a sentence in French).

many to many



Synced sequence input and output (e.g. video classification where we wish to label each frame of the video).

RNN Forward Pass

$Ux(t)$: 来自当前输入 $x(t)$ 的信息 (通过权重矩阵 U).

- The network input at time t : $W_s(t-1)$ 来自上一时刻的记忆 (隐藏状态 $s(t-1)$ 的信息通过循环权重 W).
 $h(t) = Ux(t) + Ws(t-1)$

- The activation of hidden unit at time t :

$$a(t) = h(t)$$

隐藏层的激活值
将 $a(t)$ 通过一个非线性激活函数 f 通常

- The hidden to the output at time t :

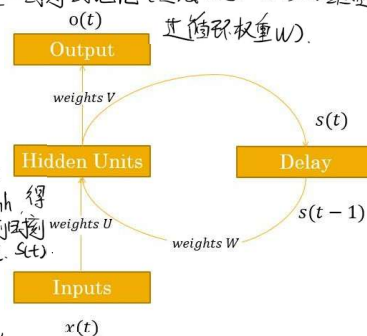
新记忆 $s(t)$ 通过
输出权重矩阵 V .

$$o(t) = Vh(t)$$

- The output of the network at time t is:

$$o(t)$$

将 $a(t)$ 通过输出激活函数, 比如 softmax, 得到此刻的最终预测值 $o(t)$.



$E(t)$: 在单个时间点 t 的误差 (均方误差).

$e(t) = d(t) - o(t)$ (真实答案 - 网络预测值).

$$E(t) = \frac{1}{2} e(t)^T e(t).$$

E_{total} (总误差): 在所有时间点, 从 t_0 到 t_1 的单个误差 $E(t)$ 的总和.

RNN Architecture

- If a network training sequence starts at time t_0 and ends at t_1 , one possible choice of the total loss function is the sum over time of the square error function

$$[e(t)]_k = [d(t)]_k - [o(t)]_k$$

$$E(t) = \frac{1}{2} e(t)^T e(t)$$

$$E_{total} = \sum_{t=t_0}^{t_1} E(t)$$

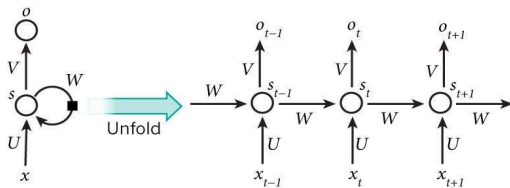
像 BP (反向传播) 这样的标准算法无法在“带循环”的网络 (如左图) 上工作。

“随时间展开/展开” (Unfolding / Unrolling) ! “展开”是一种“概念上”的转换。我们把那个带循环的 RNN (左图) , “复制”成一个没有循环的、非常深的“前馈网络” (右图) 。

展开方式:

序列中的每一个时间点 ($t-1, t, t+1, \dots$) , 都对应这个“深”网络中的一层。
例子: 如果一个句子有 5 个词, 我们就把 RNN 展开成一个 5 层的神经网络。

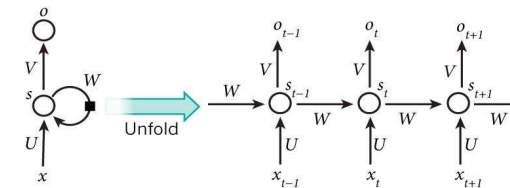
RNN Architecture



- The recurrent network can be converted into a feed forward network by unfolding over time
- The above diagram shows a RNN being unrolled (or unfolded) into a full network. By unrolling we simply mean that we write out the network for the complete sequence.
 - For example, if the sequence we care about is a sentence of 5 words, the network would be unrolled into a 5-layer neural network, one layer for each word.
- This means all the earlier theory about feed forward networks learning follows through

从 s_{t-1} 到 s_t 的 W , 和从 s_t 到 s_{t+1} 的 W 是同一个 W

RNN Architecture



- In RNN errors can be propagated more than 2 layers in order to capture longer history information.
 - This process is usually called unfolding. “循环权重 W 在所有时间步中被‘复制’ (即‘共享’) 的。”
 - In an unfolded RNN, recurrent weight is duplicated for arbitrary number of time steps

有时候并不需要每个时间点都有输出

比如情感分析 (多时) 只关心最后的 $E(t)$ 但对于短视频分类 (多时) 要 E_{total}

The above diagram has outputs at each time step, but depending on the task this may not be necessary.

For example: when predicting the sentiment of a sentence we may only care about the final output, not the sentiment after each word. Similarly, we may not need inputs at each time step. The main feature of an RNN is its hidden state, which captures some information about a sequence.

Back Propagation Through Time (BPTT)

在展开后的深层前馈网络上运行标准BP
在展开的网络上进行梯度下降。

- BPTT learning algorithm is an extension of standard backpropagation that performs gradients descent on an unfolded network.

The gradient descent weight updates have contributions from each time step.

The errors have to be back-propagated through time as well as through the network

For recurrent networks, the loss function depends on the activation of the hidden layer through its influence on the output layer **and** through its influence on the hidden layer at the next step.

当BPTT算法计算如何更新一个共享的权重(比如W)时,它会把所有时间点, $(t=1, t=2, t=3, \dots)$ 的误差贡献全部加起来, 形成一个总梯度, 然后才更新权重一次。

神经网络: 在一个时间步 t , 误差从输出层 $O(t)$ 反向传播到隐藏层 $S(t)$, 更新 S 和 O 的权重 V 。
BPTT时间: 误差从隐藏层 $S(t)$ 反向传播到上一个时刻的隐藏层 $S(t-1)$, 更新权重 W 和权重 V 。

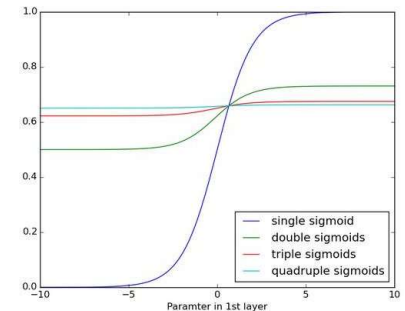
隐藏层 $S(t)$ 直接影响了当前输出 $O(t)$, 而 $O(t)$ 产生了误差 $E(t)$ 。
 $S(t)$ 通过 W 矩阵又间接影响了 $S(t+1)$, 而 $S(t+1)$ 又影响了 $O(t+1)$ 。
产生了 $E(t+1)$, 所以 $E(t)$ 的一部分也要反向传播回去, 影响 $S(t+1)$ 。
BPTT 把当前时刻和之前所有时刻产生的所有误差全部收集起来加一起算 $S(t)$ 的总梯度。

误差必须随时间反向传播, 也必须随网络反向传播。
梯度下降的权重更新来自每个时间步的贡献。

Elman (simple RNN) 无法实现长期记忆 因为 vanishing gradient.

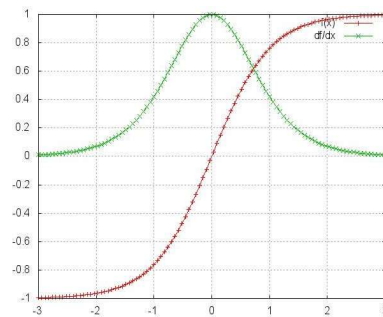
Vanishing Gradients

- RNNs trained with BPTT have difficulties learning long-term dependencies (e.g. dependencies between steps that are far apart) due to what is called the vanishing gradients problem.
- The figure shows the effects of applying a sigmoid function over and over again. The data is flattened until, for large stretches, it has no detectable slope.
- Vanishing gradients can become too small for computers to work with or for networks to learn.



Vanishing Gradients

- Sigmoid functions have derivatives of 0 at both ends
- When this happens we say the corresponding neurons are saturated
 - they have a zero gradient and drive other gradients in previous layers towards 0.
- Gradient contributions from "far away" steps become zero, and the state at those steps doesn't contribute to what you are learning
 - forget old information: end up without learning long-range dependencies.



用 BPTT 训练的 RNN，在学习“长期依赖”时会遇到困难。“长期依赖” (long-term dependencies)：就是指时间序列中，相隔很远的两个数据点之间的关联。

例子：在句子“我 30 年前出生在法国...” (此处省略 100 个词) ...所以我能说一口流利的法语。”模型必须“记住”100 个词之前的“法国”，才能在最后正确预测出“法语”。这就是“长期依赖”。问题：简单的 RNN (如 Elman 网络) 做不到这一点，它很快就会“忘记”100 个词之前发生过什么。

BPTT 算法本质是 通过链式法则把误差梯度逐层回传，从现在往回丢失。

RNN 隐藏层一般使用 sigmoid 函数或 tanh，这种 $f(x)$ 的两端曲线平坦，梯度趋于 0。

当 BPTT 算法试图把误差从 $t=100$ (“法语”) 传回 $t=1$ (“法国”) 时，它需要“穿过”100 个时间步 (即“展开后”的 100 层网络)。

在每一步，它都需要乘以当前激活函数的“梯度” (即绿线 df/dx 的值)。

梯度 (绿线) 的最大值：即使在最理想的中心点，tanh 的梯度也最多只有 1.0 (sigmoid 更惨，最多只有 0.25)。

梯度 (绿线) 的“常规”值：在“饱和区”，梯度接近 0。

梯度消失的过程：(1个<10的数) x (1个<10的数) ... 结果无限趋近于0，无法被察觉。

梯度为 0 意味着什么？

当 BPTT 算法算出 $t=11$ 处的梯度为 0 时，它等于在说：

“($t=1000$ 处的) 最终误差，与 $t=1$ 处 (“法国”) 发生了什么，完全没有关系。”

学习失败：

既然“梯度为 0”，算法就不会 (也无法) 更新 $t=1$ 时的权重。

“Gradient contributions from ‘far away’ steps become zero” (来自遥远步骤的梯度贡献变为 0)。

“forget old information” ((算法被迫) 忘记了旧信息)。

LSTM 被发明出来，就是为了解决“梯度消失”问题。LSTM 在“反向传播”过程中，能够维持一个更“恒定”的误差流。在 Elman 网络中，梯度会连乘 $0.1 * 0.1 * \dots$ 然后“消失”；而在 LSTM 中，它被设计为可以像 $1.0 * 1.0 * \dots$ 一样流动，不会消失。Elman 网络（简单 RNN）的梯度传不回“法国”；而 LSTM 的梯度（误差）可以轻松地从“法语”一路“恒定地”传播回 100 个词之前的“法国”，从而让网络学会这个“长期”的关联。

LSTM - introduction

"LSTM - same idea as RNN" (LSTM 和 RNN 是同样的想法)

- LSTM was invented to solve the vanishing gradients problem.
- LSTM maintain a more constant error flow in the back propagation process.
- LSTM can handle large sequences that are linked remotely.

LSTM – same idea as RNN

它“吃”进 2 个东西（输入）：

x_t ：当前时刻的输入（比如，当前的词“法语”）。

s_{t-1} ：上一刻的“状态”（即“旧记忆”，比如 $t-1$ 时刻的“思考结果”）。

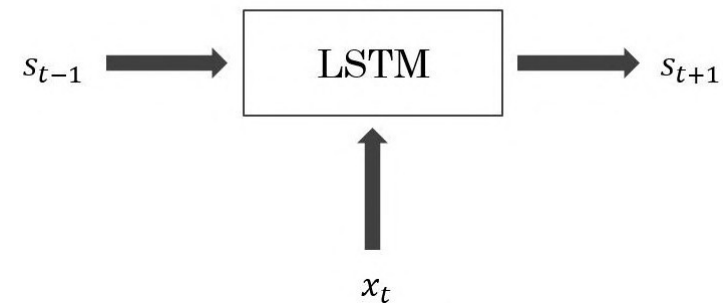
y_t

它“吐”出 2 个东西（输出）：

y_t ：当前时刻的输出（比如，对下一个词的“预测”）。

s_t ：当前时刻的“新状态”（即“新记忆”，它将被传递到 $t+1$ 时刻）。

（注：图中的 s_{t+1} 应该是指“将被传递到下一刻”的状态，我们通常把它记为 s_t 。）



Elman 网络 (RNN): 隐藏层是一个“简单”的单元 (比如一个 $\tanh$ 激活函数)。我们已经知道, 这种简单结构在 BPIT 训练中会导致“梯度消失”。

LSTM 网络: 它“替换”了 RNN 的那个“简单”隐藏单元, 取而代之的是一个“复杂的”、“内存块” (memory block)。这个“内存块”的核心是“记忆细胞” (memory cell) 和“自适应门控单元” (adaptive gating units) (即输入门和输出门)。

LSTM的隐藏层被一个“内存块” (Memory Block) 取代了。

这个“内存块”的内部要复杂得多, 它包含:

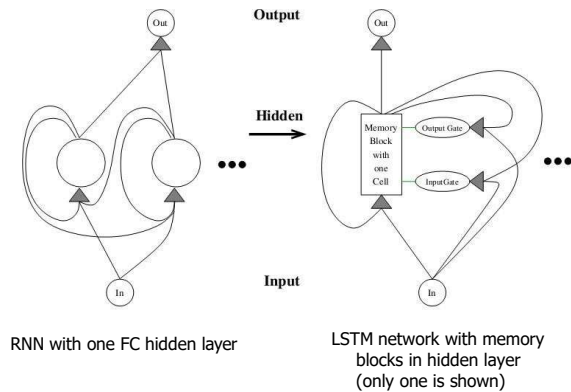
“记忆细胞” (Memory Cell): 真正存储“长期记忆”的地方 (即我们之前讨论的 $c(t)$ 或 CEC)。

“输入门” (Input Gate): 控制“新信息”何时可以“写入”记忆。

“输出门” (Output Gate): 控制“内部记忆”何时可以“读取”并影响输出。

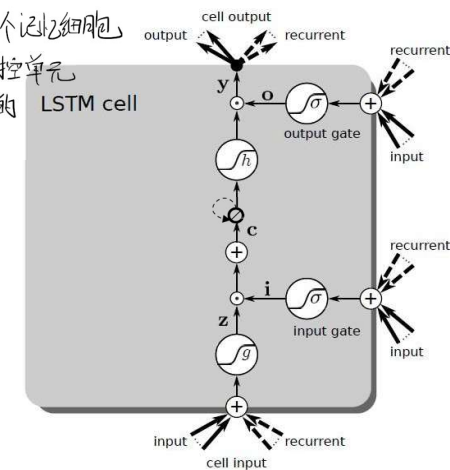
LSTM Architecture

- The basic unit in the hidden layer of an LSTM network is a memory block, it replaces the hidden unit in a traditional RNN.
- A memory block contains one or more memory cell and a pair of adaptive multiplicative gating units which gates input and output to all cells in the block.
- Memory blocks allow cells to share the same gates thus reducing the number of parameters.



LSTM Architecture - overview

一个内存块包含一个或多个记忆细胞
和一对自适应的乘法门控单元
这些门控制着对细胞的
输入和输出。



自适应门控: 门是可以学习的。
 $i(t)=1$ 时开门, $i(t)=0$ 时关门
通过未计算
· 写入记忆: $z(t) \odot i(t)$
· 读取记忆: $h(c(t)) \odot o(t)$
内存块允许多个细胞共享
同一套门, 从而减少参数数量

LSTM Architecture

- input

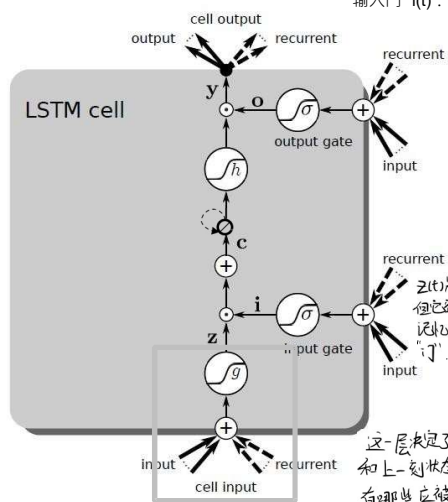
输入(input) → 计算候选信息 $z(t)$

LSTM 单元如何决定“写入”什么新信息到它的“长期记忆” $c(t)$ 中。这个“写入”操作被分成了两个独立的部分：

“候选信息” $z(t)$ ：我们“可能”想写什么？

$z(t)$ 代表了“基于当前输入，我们新生成的、可能要存入记忆的想法”。

“输入门” $i(t)$ ：我们“允许”写入多少？



- The layer decides which information from the input should be forwarded into the LSTM cell. g 通常是一个 tanh 激活函数，它把 $z(t) = g(a_z(t))$ 输入 $a_z(t)$ 压缩到 -1 到 1 之间，生成候选信息 $z(t)$ 。
- By multiplying $y(t-1)$ with the recurrent weight matrix R_z , the layer decides which information from the previous time step should be forwarded into the LSTM cell. $z(t)$ 是新想法，但它不能直接进入记忆，必须先通过“门”。

$$a_z(t) = W_z x(t) + R_z y(t-1)$$

$z(t) = g(a_z(t))$
这一层决定了来自当前输入 $x(t)$ 和上一时刻状态 $y(t-1)$ 的信息中，有哪些应该被写入 LSTM 单元。
 $W_z x(t)$ 来自当前时间点的输入 $x(t)$ 。
 $R_z y(t-1)$ 来自上一时间点的循环状态 $y(t-1)$ 。

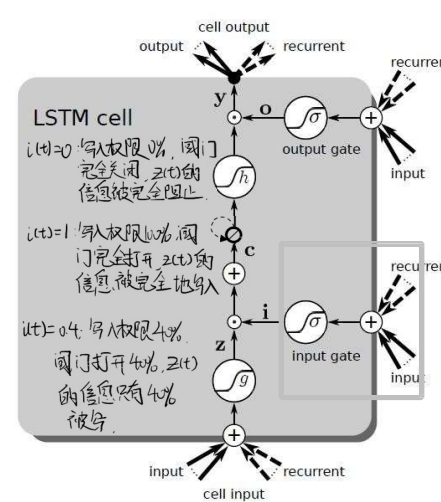
“输入门” $i(t)$ 的计算： $i(t)$ 是一个“阀门”，它决定了 $z(t)$ 这个“新想法”有多大比例（百分之几）可以被“允许”写入“长期记忆”。输入门控制着对“记忆细胞”的“写入权限”。

LSTM Architecture

- input gate

注意，在计算公式中，输入门也查看当前输入 $x(t)$ 和上一时刻状态 $y(t-1)$ ，但它使用一套独立的权重 W_{in} 和 R_{in} 。

$i(t) = G(a_{in}(t))$ ， G 是 sigmoid 函数，输出值总在 0~1 之间。



- The input gate controls write accesses to memory cells.
- This is realized by using the result of the squashing function σ as a factor which will be later multiplied by the squashed cell input $z(t)$.
- The range of function σ is (0,1) and can be interpreted in case of 0 as write access denied and in case of 1 as write access granted.
- Note that all values in between are also possible.

$$a_{in}(t) = W_{in} x(t) + R_{in} y(t-1)$$

$$i(t) = \sigma(a_{in}(t))$$

每个“记忆细胞”都包含一个节点，这个节点带有一个权重固定为1的“自我循环”连接

LSTM Architecture – CEC

恒误差流
CEC解决了梯度消失问题

$$C(t) = z(t) \odot i(t) + c(t-1)$$

$\odot$: Hadamard 积 (逐元素相乘)

BPTT 算法在反向计算梯度时，需要计算 $C(t)$ 对 $C(t-1)$ 的偏导数

由于这是一个加法 $(A+B)$, $C(t-1)$ 这一项 (B) 的梯度 (导数) 永远是1.

CEC 的数字形式是加法，反向传播梯度恒为1，其中梯度不随时间而衰减，因此可以传递很远。

Each memory cell contains a node with a self-connected recurrent edge of fixed weight one, ensuring that the gradient can pass across many time steps without vanishing – which is called CEC (constant error carousel)

LSTM (CEC) 梯度 (反向) 是: 梯度 $(1-1) =$ 梯度 $(t) \times 1$ (在加 $\rightarrow$ 恒定)

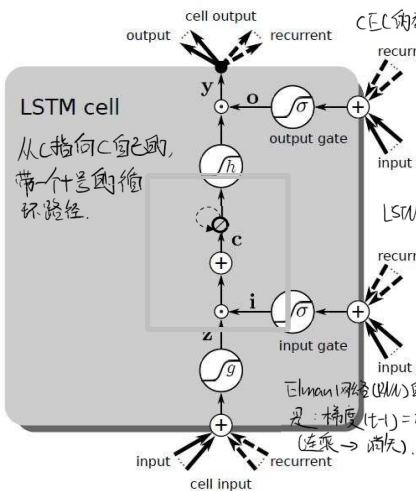
The CEC solves the vanishing error problem. In the absence of new input or error signals to the cell the CEC local error back flow remains constant, neither growing or decaying.

Elman 网络 (RNN) 的梯度 (反向)

$$C(t) = z(t) \odot i(t) + c(t-1)$$

是: 梯度 $(t-1) =$ 梯度 $(t) \times$ 权重 (连接 $\rightarrow$ 权重)

$\odot$ is the Hadamard product

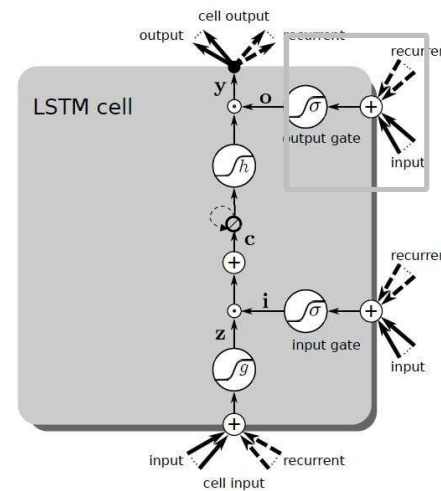


LSTM Architecture – output gate

LSTM 单元如何决定“对外输出”什么信息。这个“输出”操作和我们之前看的“写入”操作一样，也是一个“两步走”的过程：

“输出门” $o(t)$ ：我们“允许”对外输出多少信息？（计算“阀门”大小）

“最终输出” $y(t)$ ：结合“阀门”和“内部记忆”，计算出最终的输出。

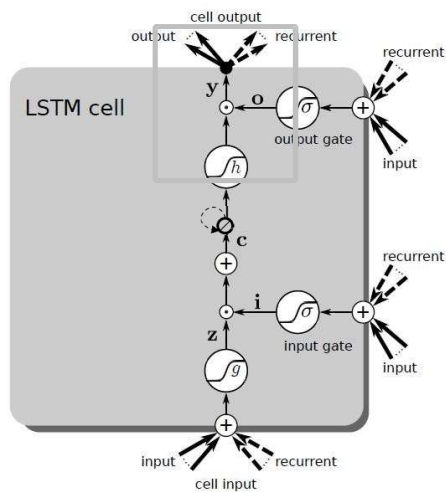


- The output gate controls read accesses from memory cells.
- This is realized by using the result of the squashing function σ as a factor which will be later multiplied by the squashed cell content $h(c(t))$.
- The range of function σ is $(0,1)$ and can be interpreted in case of 0 as read access denied and in case of 1 as read access granted.
- Note that all values in between are also possible.

$$a_{out}(t) = W_{out}x(t) + R_{out}y(t-1)$$

$$o(t) = \sigma(a_{out}(t))$$

LSTM Architecture – output



- The value stored in the cell $c(t)$ is squashed by function h , and whether this information gets output is decided by the output gate via $o(t)$.

$$y(t) = h(c(t)) \odot o(t)$$

$\odot$ is the Hadamard product

“输出门”(Output Gate) -> 计算“读取阀门” $o(t)$

“输出门” $o(t)$ 的计算, $o(t)$ 是一个“阀门”, 它决定了我们“长期记忆” $c(t)$ 中的内容, 有多大比例 (百分之几) 可以被“允许”在此刻“读取”并发送到外部 (作为 $y(t)$)。

所以说, 输出门控制着从“记忆细胞”中“读取”的权限。

$O_{out}(t) = W_{out}x(t) + R_{out}y(t-1)$
和输入门一样, 它也查看当前输入 $x(t)$ 和上一时刻状态 $y(t-1)$, 并使用一套独立的权重 W_{out} 和 R_{out} 。

$O(t) = \sigma(O_{out}(t))$ σ 是 sigmoid 函数, 在 0 到 1 之间

$i(t)=0$: 读取权限 0%, 阀门完全关闭, LSTM 决定完全隐藏它的内部记忆, 什么都不输出 ($y(t)$ 将会是 0)

$i(t)=1$: 读取权限 100%, 阀门完全打开, LSTM 决定完全暴露它的内部记忆

$i(t)=0.4$: 读取权限 40%, $O(t)$ 是一个读取阀门, 控制不输出, 用来控制输出。

“输出”(Output) -> 计算“最终输出” $y(t)$

“最终输出” $y(t)$ 的计算, $y(t)$ 就是 LSTM 单元在 t 时刻真正对外输出的值 (比如, 对下一个词的预测)。
存储在细胞 $c(t)$ 中的值首先被函数 h “压缩”, 然后, 这个信息是否被允许输出, 由“输出门” $o(t)$ 来决定。

$$y(t) = h(c(t)) \odot o(t)$$

$h(c(t))$: h 通常是 $\tanh$, $h(c(t))$ 是把长期记忆 $c(t)$ “压缩”到 1 之间, 使得到一个准备好被输出的内部状态。

$O(t)$: 上一步算出来的读取阀门 (0~1 之间)。

$y(t)$: 最终输出 = (压缩后的内部记忆) $\times$ 读取阀门

假设“内部记忆” $h(c(t))$ 是 $[1.0, 0.8]$ 。

假设“输出门” $o(t)$ 学会了只输出第二部分, 所以 $o(t)$ 是 $[0, 1]$ 。

最终输出 $y(t) = [1.0, 0.8] \odot [0, 1] = [0, 0.8]$ 。

结果: LSTM 成功地“隐藏”了它的第一部分记忆, 只“选择性地”输出了第二部分。

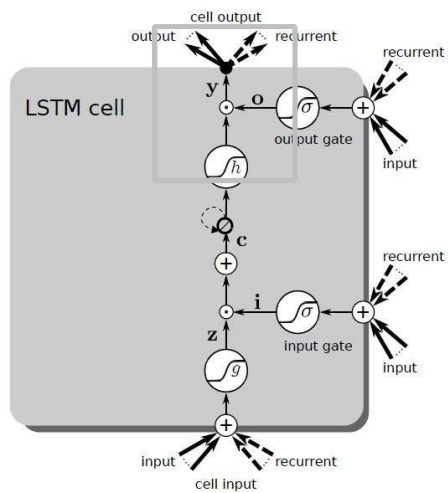
LSTM 的“输出”机制是一个“两步走”的“受控读取”过程:

计算“阀门”: 网络根据当前输入“和”旧记忆”, 计算出一个“输出门” $o(t)$ (0 到 1 之间)。

计算“输出”: 网络取出“长期记忆” $c(t)$, 用 $\tanh$ 压缩它, 然后乘以 $o(t)$ 这个“阀门”, 得到最终的输出 $y(t)$ 。

$y(t)$ 既是对外的“预测”, 也会在下一刻 $t+1$ 作为“循环状态” (即 $y(t-1)$) 被反馈回来, 用于计算 $t+1$ 时刻的“门”和“候选信息”。

LSTM Forward Pass



- The cell state is updated based on its current state and 3 inputs: , ,

$$a_z(t) = W_z x(t) + R_z y(t-1), z(t) = g(a_z(t))$$

$$a_{in}(t) = W_{in} x(t) + R_{in} y(t-1), i(t) = \sigma(a_{in}(t))$$

$$c(t) = z(t) \odot i(t) + c(t-1)$$

$$a_{out}(t) = W_{out} x(t) + R_{out} y(t-1), o(t) = \sigma(a_{out}(t))$$

$$y(t) = h(c(t)) \odot o(t)$$

1. 计算候选信息 $z(t)$.

$$\text{公式: } a_z(t) = W_z x(t) + R_z y(t-1), z(t) = g(a_z(t))$$

网络首先根据当前输入 $x(t)$ 和上一时刻的输出 $y(t-1)$ (即上下文) 计算出一个候选信息 $z(t)$. 我们想记住的新想法 (g 通常是 $\tanh$).

2. 计算输入门 $i(t)$.

$$\text{公式: } a_{in}(t) = W_{in} x(t) + R_{in} y(t-1), i(t) = \sigma(a_{in}(t))$$

同时, 网络计算输入门 $i(t)$ 的值. 这是一个 0 到 1 之间的写入阀门, 它决定了 $z(t)$. 这个新想法有多大比例可以被写入记忆. (σ 是 sigmoid).

3. 更新细胞状态 (长期记忆 $c(t)$)

$$\text{公式: } c(t) = z(t) \odot i(t) + c(t-1)$$

CRC 的核心: 新记忆 $c(t) = (\text{候选信息 } z(t) \times \text{输入门 } i(t)) + \text{旧记忆 } c(t-1)$

4. 计算输出门 $o(t)$.

$$\text{公式: } a_{out}(t) = W_{out} x(t) + R_{out} y(t-1), o(t) = \sigma(a_{out}(t))$$

网络计算输出门 $o(t)$ 的值. 0~1 之间的读取阀门. 决定长期记忆 $c(t)$ 有多大比例可以被对外输出.

5. 计算最终输出 $y(t)$.

$$\text{公式: } y(t) = h(c(t)) \odot o(t)$$

h 通常是 $\tanh$. 首先将长期记忆 $c(t)$ 压缩到 -1 到 1 之间.

然后这个压缩后的记忆乘以读取阀门 $o(t)$. 得到当时时刻的最终输出 $y(t)$.

这个 $y(t)$ 既是对外的预测, 也会在 T -时刻 $t+1$ 作为 $y(t+1)$ 被反馈回来, 开始新一轮计算.

当 BPTT 算法在反向传播误差时 (从 t 时刻传到 $t-1$ 时刻), 误差信号会“抵达”CEC (即 $c(t)$).

因为 CEC 的数学本质是“加法”, 它的导数 (梯度) 恒定为 1. $c(t) = new + c(t-1)$

这使得误差信号可以在 CEC 内部停留很长时间, 并“不衰减地” (“non-decaying”) 被传播回非常遥远的过去.

LSTM Backward Pass

- Errors arriving at cell outputs are propagated to the CEC
- Errors can stay for a long time inside the CEC
- This ensures non-decaying error
- Can bridge time lags between input events and target signals

Advantages of LSTM

不衰减的误差反向传播

- Non-decaying error backpropagation.

长周期的记忆

- For long time lag problems, LSTM can handle noise and continuous values.

LSTM 可以处理那些“过去”和“现在”相隔很远的问题.

- No parameter fine tuning.

无需参数微调

- Memory for long time periods

Limitations of LSTM

如果你持续向一个序列输入信息（比如读一篇很长的小说）， $c(t)$ 的值会不断地“累加”，导致 $c(t)$ 变得“无限大”（“unbounded fashion”）。

无限大会导致输出端的压缩函数 $h(c(t))$ “饱和”

细胞状态 $c(t)$ 倾向于“线性增长”

$c(t) = z(t) \odot i(t) + c(t-1) \rightarrow$ grows linearly

- LSTM allows information to be stored across arbitrary time lags and error signals to be carried far back in time.
- This potential strength however, can contribute to a weakness: the cells states $c(t)$ often tend to grow linearly during the presentation of a time series.
- If we present a continuous input stream, the cell state may grow in unbounded fashion causing saturation of the output squashing function $h(c(t))$.

Conclusions

- RNNs - self connected networks
- Vanishing gradients and long memory problems
- LSTM - solves the vanishing gradients and the long memory limitation problem

